Software Requirements Specification

Project: [Working Title] — Shared Event Photo Book

Version: 0.1 (Draft) **Status:** In progress

---

1. Introduction

1.1 Purpose

This document specifies the requirements for a web application that lets people share photos from shared experiences (events, trips, gatherings) privately with the people who were actually there — rather than broadcasting to a public feed. Tagged individuals see the photo appear in their own personal gallery only after they approve it.

1.2 Scope

The system will:

- Allow users to create "events" and upload photos to them
- Allow anyone tagged/invited to an event to also upload photos to it
- Allow members to tag other users in the event itself (not per individual photo)
- Require tagged users to approve the tag before the event's photos appear in their personal gallery
- Provide a web interface to browse photos within an event

The system will explicitly **not** include public profile pages or a public feed, or any messaging/DM functionality, in this version.

1.3 Intended Audience

General public — no niche technical audience assumed. UI and flows should not assume technical literacy.

1.4 Definitions

Term	Meaning
Event	A container for a set of photos tied to a shared experience (e.g. a trip, a party)

Term	Meaning
Tag	An association between an event and a user, indicating they were part of that event
Approval	The action a tagged user takes to accept a tag, causing the photo to enter their gallery
Gallery	A user's personal collection of approved, tagged photos
Member	A user who has been invited to / is part of a given event

1.5 Language

All UI copy and documentation: English.

---

2. Overall Description

2.1 Product Perspective

A new, standalone web application, not integrated with any other existing project. Hosted on infrastructure the team controls, but not self-hosted on personal hardware — a managed hosting setup.

2.2 User Classes

- **Registered user** — has an account, can create events, upload photos, tag others, approve/reject tags of themselves.
- No admin/moderator role for this version — the system is fully peer-to-peer with no moderation layer.
- `[Future consideration]` Photo encryption is planned for a future version, not this one — worth keeping in mind for storage architecture decisions now even if not implemented yet.

2.3 Operating Environment

- Backend: Go, using the Gin framework
- Frontend: React
- Not self-hosted by end users — hosted centrally by the project owners

2.4 Constraints

- No public-facing pages (no public profiles, no public event pages, no discoverability outside invited members)

- No in-app messaging in this version
- No third-party integrations in this version

2.5 Assumptions

- Users authenticate with individual accounts (method TBD — see open questions)
- An "event" is the core organizing unit; there is no ungrouped/global photo stream

---

3. Functional Requirements

3.1 Accounts & Authentication

- **FR-1:** The system shall allow a person to register for an account using a locally stored username and password. *(OAuth with Google is planned for a future version, not v1.)*
- **FR-2:** The system shall allow a registered user to log in and log out.
- **FR-3:** The system shall allow a registered user to search for other registered users (e.g. by username), so they can be invited to events or tagged.

3.2 Events

- **FR-4:** The system shall allow a registered user to create an event.
- **FR-5:** The system shall allow the event creator to invite other registered users to the event as members.
- **FR-6:** The system shall allow any member of an event (not just the creator) to upload photos to that event.
- **FR-7:** The system shall allow a member to view all photos within an event they belong to, in a scrollable view.
- **FR-8:** Users who are not members of an event shall not be able to view its contents. *(no public pages)*

3.3 Tagging & Approval

- **FR-9:** The system shall allow a member of an event to tag another user in that event (i.e. mark them as having been part of it).
- **FR-10:** The system shall notify a user in-app when they have been tagged in an event. *(In-app only for the web version — no email or push notifications in this version.)*
- **FR-11:** An event's photos shall **not** appear in the tagged user's personal gallery until that user explicitly approves the tag for that event.
- **FR-12:** The system shall allow a tagged user to reject a tag, in which case the event's photos do not appear in their gallery and the tagger is not able to re-tag them without the

tagged user's consent. The tagger shall be notified in-app when a tag they placed is rejected.

- **FR-13:** Upon approval, all of the event's photos (including any added later, while the user remains an approved member) shall appear in the tagged user's personal gallery.
- **FR-14:** The system shall allow a user to remove an approved event tag from their own gallery at any time after the fact.

3.4 Personal Gallery

- **FR-15:** The system shall provide each user a personal gallery view showing all photos they have approved tags for, aggregated across events.
- **FR-16:** The event creator shall give the event a name at creation time, and events shall be searchable by that name.
- **FR-17:** The gallery shall provide a date-sorted view of events, newest to oldest, that the user can scroll through.

---

4. Non-Functional Requirements

- **NFR-1 (Privacy):** No photo or event data shall be accessible to a user who is not a member of that event. This is a core product differentiator and should be treated as a hard constraint, not just a default.
- **NFR-2 (Simplicity):** Per project intent, the system shall favor minimal features and simple flows over configurability — avoid scope creep into social-network territory (comments, likes, feeds, DMs).
- **NFR-3 (Volume limits):** An event shall support a maximum of 10 photos/videos.
- **NFR-4 (Data ownership):** Users shall be able to export their own data. However, an event's data shall remain on the server as long as at least one member remains part of that event — a single member's departure (or account deletion) does not delete the event or its photos for the remaining members.
- **NFR-5 (Image handling):** Images shall be compressed on upload. Storage shall use S3-compatible object storage rather than filesystem storage.

---

5. External Interface Requirements

None at this stage — no third-party integrations, no external APIs consumed or exposed. `[Future consideration]` The backend may expose APIs for mobile apps in a later version; not part of this scope.

6. Other Requirements

- **Legal/compliance:** As stated in NFR-4, an event's photos remain on the server as long as at least one member is still part of that event. A single member leaving or deleting their account does not trigger deletion of the event or its photos. A privacy policy covering this should still be published to users at signup, even for an MVP, since photos may include people who did not upload them themselves.